

10 More Code Projects to Learn Claude Effectively

A follow-up set of projects: five that move beyond microcontrollers, and five focused on the Windows / Visual Studio ecosystem.

Part 1: General Software Projects (Non-Microcontroller)

1. Personal Knowledge-Base CLI

Build a Python command-line tool that ingests your notes, PDFs, and bookmarks into a local SQLite + embeddings store, then lets you query them in natural language via the Claude API. You'll have to make real decisions about chunking, retrieval, prompt construction, and how to display citations — the kind of design choices you can't get from a tutorial.

The lesson: *Retrieval-augmented generation from first principles.* You'll see exactly where Claude shines and where bad retrieval poisons its answers.

2. Static Analyzer for Your Own Python Code

Write a tool that walks the AST of a Python project, flags suspicious patterns (mutable default args, swallowed exceptions, dead code), and then asks Claude to explain or suggest fixes for each finding. The interesting part is the *handoff* — you decide what context Claude needs (the function body? the call sites? the test?) before it can be useful.

The lesson: "Send more context" is not always the right answer. You'll learn to curate context deliberately.

3. Compiler / Interpreter for a Toy Language in C++

Pick a small language (a Lisp subset, a stack VM, a TCL-like shell) and implement it in modern C++ with Claude pairing. Use Claude for the boring parts (tokenizer, AST visitor boilerplate) and the hard parts (figuring out why your garbage collector segfaults under recursion).

The lesson: *Separate generation from reasoning.* Claude is great at writing the lexer in two minutes, but you'll learn when to stop generating and start reading code together line by line.

4. Differential Testing Harness

Build a Python harness that takes two implementations of the same spec (your code vs. a reference, two versions of a refactor, Python vs. a C++ port) and fuzzes them with property-based tests to find behavioral divergence. Use Claude to generate the property generators and to triage failing cases.

The lesson: Using Claude to *design tests you wouldn't have thought of* is one of the highest-leverage uses of a model on real codebases.

5. Git History Archaeologist

Write a Python tool that ingests a repo's `git log`, diffs, and issue tracker, then uses Claude to answer questions like "why was this function written this way?" or "who introduced this regression and what were they trying to fix?"

The lesson: How to manage Claude's context window against a corpus that doesn't fit, how to design summarization passes that preserve signal, and how to cite back to a specific commit so answers are verifiable rather than plausible-sounding.

Part 2: Windows-Focused Projects

6. Win32 System Tray Utility in C++

Build a small native tray app — a clipboard history manager, a CPU/RAM mini-monitor, or a "focus mode" that mutes notifications. You'll wrestle directly with the Win32 message loop, `Shell_NotifyIcon`, and resource files.

The lesson: *Navigating sprawling, decades-old APIs.* You'll learn how to prompt for "the modern, supported way to do X on Windows 11" rather than accepting the first MSDN-style snippet, and how to verify against the actual Microsoft docs.

7. WPF or WinUI 3 Dashboard App in C#

Pick a dataset you care about (Steam library stats, electric bill history, GitHub activity) and build a desktop dashboard with charts, filtering, and data binding. Use Claude for XAML — which it generates competently but often verbosely — and have it review your MVVM separation.

The lesson: *Iterating on UI code with a model that can't see the rendered result.* You'll learn to describe layout problems precisely and to send screenshots back when words aren't enough.

8. PowerShell Automation Suite

Write a set of PowerShell scripts that automate your actual Windows workflow: bulk-renaming downloads, cleaning out %TEMP%, setting up a fresh dev machine, exporting installed apps to a manifest. Have Claude help convert one-off commands into idempotent, parameterized, error-handling scripts.

The lesson: *Claude as a translator between "what I want" and a language you only half-know.* PowerShell's verb-noun conventions and pipeline semantics are exactly the kind of thing Claude is excellent at scaffolding while you learn.

9. Visual Studio Extension (VSIX) in C#

Build a real extension — a custom code-snippet inserter, a TODO-comment scraper that opens a tool window, or a build-time stopwatch. The VSIX SDK is famously underdocumented and full of MEF/COM cruft.

The lesson: *How to give Claude enough context about a poorly-documented framework* — pasting in package manifest snippets, interface signatures from `Microsoft.VisualStudio.Shell`, and your actual error dialogs — and how to recognize when it's confidently making things up.

10. Native File Search Tool (C++ or C#)

Build a "better than Windows Search" desktop utility: walk NTFS, index filenames (or use the USN journal for instant updates), expose a hotkey-triggered search box, and rank results by recency and usage.

The lesson: *Performance, threading, and OS-level APIs.* You'll learn to catch suggestions that compile but would lock up the UI thread or thrash the disk, and to ask Claude to reason about cost rather than just correctness.

How to Pick

Start with a project that scratches a real itch — you'll push back harder when Claude's suggestions don't fit your actual constraints, and that pushback is where the learning happens.

Acronym List

- API** — Application Programming Interface
- AST** — Abstract Syntax Tree
- CLI** — Command-Line Interface
- COM** — Component Object Model
- MEF** — Managed Extensibility Framework
- MSDN** — Microsoft Developer Network
- MVVM** — Model-View-ViewModel
- NTFS** — New Technology File System
- RAG** — Retrieval-Augmented Generation
- SDK** — Software Development Kit
- TCL** — Tool Command Language
- UI** — User Interface
- USN** — Update Sequence Number
- VM** — Virtual Machine
- VSIX** — Visual Studio Extension (package)
- WPF** — Windows Presentation Foundation
- WinUI** — Windows UI Library
- XAML** — Extensible Application Markup Language